



Prática - 1: Interrupções e chamadas de Sistema

Professor: Rogério Pereira Junior

rogerio.pereira@ifsc.edu.br

Observações Gerais

- Cada atividade deve ser acompanhada de uma **captura de tela** (screenshot) mostrando a execução do comando solicitado.
- O relatório deve conter uma **descrição clara das observações**, bem como respostas às questões propostas em cada item.

Revisão Conceitual Profunda

Antes de iniciar a prática, é importante revisar os três pilares que conectam o código em **Modo Usuário** ao **Kernel** e ao **Hardware**. Esses conceitos fornecem a base para compreender o funcionamento dos utilitários `strace` e `ltrace`.

Chamada de Sistema (Syscall / Interrupção de Software / Trap)

- **O que é:** Um pedido formal feito por uma aplicação em Modo Usuário (Ring 3) para que o Kernel (Ring 0) execute uma operação privilegiada em seu nome (como acesso a disco, rede, criação de processos).
- **Como funciona:** A aplicação preenche registradores com os parâmetros e o código da chamada (opcode) e dispara uma instrução especial (`syscall` em `x86_64` ou `sysenter` em `x86_32`). Essa instrução alterna a CPU para o Modo Núcleo e salta para a rotina tratadora do Kernel.

Interrupções de Hardware (IRQ - Interrupt ReQuest)

- **O que é:** Sinais elétricos enviados assincronamente por periféricos (placa de rede, controlador de disco, teclado, temporizador) ao processador para indicar que um evento externo ocorreu e exige atenção.
- **Como funciona:** Ao receber uma IRQ, a CPU interrompe o programa em execução, salva seu estado atual, consulta a Tabela de Vetores de Interrupção (IVT) e executa o *Interrupt Handler* associado ao dispositivo. Em seguida, devolve o controle ao programa interrompido.

Exceções (Traps / Faults de CPU)

- **O que é:** Eventos síncronos gerados pela própria CPU quando tenta executar uma instrução ilegal (como divisão por zero, opcode inválido ou violação de acesso à memória detectada pela MMU).
- **Resultado:** O Kernel intercepta a exceção e, em geral, encerra o processo infrator (exemplo: `Segmentation Fault`).

Experimento 0: Instalação de um SO - Virtualização

Faça a instalação de um sistema operacional Linux (Ubuntu, Linux Mint ou Ubuntu MATE) no VirtualBox com as seguintes características:

- Nome da VM: SOP-2026/2
- **login:** aluno
- **senha:** aluno
- 4 processadores
- 4GB de RAM
- 30 GB de espaço em disco;

Para poder utilizar o sistema operacional virtualizado em tela cheia pode ser necessário recorrer ao interpretador de comandos

- Pesquise no menu pelo programa chamado **terminal**
- Com ele aberto, digitamos:
- **sudo apt-get update**
- **sudo apt-get install build-essential module-assistant dkms**
- **sudo m-a prepare**
- Após esta instalação, podemos clicar em Dispositivos
- Ir na opção Inserir imagem de CD dos adicionais para convidado e executar a aplicação que irá abrir.
- Reinicie a maquina (virtual) e veja a possibilidade de ampliar a tela

Experimento 1: Introduzindo comandos

1. O utilitário **strace** do UNIX permite observar a sequência de chamadas de sistema efetuadas por uma aplicação. Em um terminal, execute **strace date** para descobrir quais os arquivos abertos pela execução do utilitário date (que indica a data e hora correntes). Por que o utilitário date precisa fazer chamadas de sistema?
2. O utilitário **ltrace** do UNIX permite observar a sequência de chamadas de biblioteca efetuadas por uma aplicação. Em um terminal, execute **ltrace date** para descobrir as funções de biblioteca chamadas pela execução do utilitário date (que indica a data e hora correntes). Pode ser observada alguma relação entre as chamadas de biblioteca e as chamadas de sistema observadas no item anterior?

Experimento 2: A "Mágica"por trás do printf (ltrace vs strace)

- Crie o arquivo lab1.c:

```
#include <stdio.h>

int main() {
    printf("Sistemas Operacionais - Aula Pratica\n");
    return 0;
}
```

- Compile: gcc lab1.c -o lab1
- Intercepte as chamadas de biblioteca com ltrace:

```
ltrace ./lab1
```

- Intercepte as chamadas de sistema (syscalls) com strace:

```
strace ./lab1
```

CAPTURA DE TELA: Saída do strace ./lab1, destacando a chamada de sistema write(1, "Sistemas Operacionais...", ...).

Explique com suas palavras a diferença entre o que o comando ltrace intercepta e o que o strace intercepta. Por que o printf() precisa disparar a syscall write()?

Experimento 3: Transpondo a Barreira de Privilégio (Exceção de Hardware)

Conforme discutido em sala, a MMU e a CPU protegem os endereços do sistema. Se uma aplicação tentar acessar uma área proibida, uma Exceção de Proteção é disparada pela CPU.

1. Crie o arquivo excecao.c:

```
#include <stdio.h>

int main() {
    int *p = NULL; // Ponteiro aponta para o endereço zero (protegido)
    printf("Tentando escrever em endereço de memória proibido (NULL)...\n");
    *p = 999;      // Provoca exceção de hardware gerada pela MMU
    return 0;
}
```

2. Compile e execute:

```
gcc excecao.c -o excecao && ./excecao
```

CAPTURA DE TELA: O erro Segmentation fault (core dumped) no terminal.

Pesquise, qual componente de hardware detecta a tentativa de acesso indevido à memória e como o Kernel reage ao receber essa exceção

Experimento 4: Falando Diretamente com o Kernel (Syscall Direta)

Na arquitetura x86_64 no Linux, a Syscall de código 1 é o SYS_write. Podemos ignorar as rotinas normais do printf e disparar a instrução de transição para o Kernel manualmente

1. Crie o arquivo direto.c:

```
#include <unistd.h>
#include <sys/syscall.h>

int main() {
    char msg[] = "Escrita realizada via Syscall direta sem printf!\n";

    // syscall(código_da_syscall, file_descriptor, ponteiro_buffer, tamanho_bytes)
    // SYS_write = opcode 1 no Linux x86_64
```

```
// 1 = file descriptor para saída padrão (stdout)
syscall(SYS_write, 1, msg, sizeof(msg) - 1);

return 0;
}
```

2. Compile e execute acompanhando com strace:

```
gcc direto.c -o direto
strace ./direto
```

CAPTURA DE TELA: Saída do strace ./direto mostrando a chamada de sistema ocorrendo diretamente.

Experimento 5: Observação de Interrupções

Agora vamos gerar rajadas de chamadas de sistema e operações de disco e acompanhar, em tempo real, o aumento no número de interrupções por segundo (in)

- Abra dois terminais lado a lado.
- No Terminal A: Inicie o monitoramento em tempo real:

```
vmstat 1
```

Observe os valores médios da coluna in no sistema em repouso (baseline).

- No Terminal B: Crie e execute um programa em C chamado gerador_io.c que força o Kernel a realizar operações de entrada/saída contínuas:

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    printf("Gerando carga de E/S e Syscalls... Pressione Ctrl+C para parar.\n");
    int fd = open("/dev/null", O_WRONLY);
    char buffer[1024];

    while(1) {
        // Executa chamadas repetidas de escrita para forçar tração no Kernel
        write(fd, buffer, sizeof(buffer));
    }
    return 0;
}
```

- Compile e execute no Terminal B:

```
gcc gerador_io.c -o gerador_io && ./gerador_io
```

- Observe o Terminal A: Note o salto instantâneo nas colunas in (interrupções).
 - Qual era o valor aproximado da coluna in (interrupções/seg) com o sistema em repouso?
 - Para quanto esse valor subiu durante a execução do loop de E/S?
- Aprofundando no /proc/interrupts: No Terminal B, pare o programa (Ctrl+C) e execute:

```
cat /proc/interrupts | grep -E "CPU0|SATA|nvme|timer|LOC"
```

CAPTURA DE TELA: Captura do terminal dividido mostrando o vmstat 1 reagindo à execução do gerador_io (evidenciando o pico nos valores da coluna in)

Experimento 6: Investigando os Mapeamentos do Hardware

Em aula vimos uma tabela que mostra como portas de E/S e endereços são atribuídos a controladores de dispositivos. O Kernel do Linux expõe essa estrutura na árvore do sistema de arquivos virtual **/proc**.

- (a) Execute o comando para ver as portas de entrada e saída ativas:

```
cat /proc/ioports | head -n 25
```

- (b) Execute o comando para ver os canais de DMA (Acesso Direto à Memória) alocados:

```
cat /proc/dma
```

CAPTURE DE TELA: Saída do comando `/proc/ioports` exibindo a alocação dos controladores no hardware.

Perguntas finais

1. Quais são as duas principais funções de um sistema operacional?
2. Instruções relacionadas ao acesso a dispositivos de E/S são tipicamente instruções privilegiadas, isto é, podem ser executadas em modo núcleo, mas não em modo usuário. Dê uma razão de por que essas instruções são privilegiadas.
3. Qual é a diferença entre modo núcleo e modo usuário? Explique como ter dois modos distintos ajuda no projeto de um sistema operacional.
4. Quais das instruções a seguir devem ser deixadas somente em modo núcleo?
 - (a) Desabilitar todas as interrupções.
 - (b) Ler o relógio da hora do dia.
 - (c) Configurar o relógio da hora do dia.
 - (d) Mudar o mapa de memória
5. Quando um programa de usuário faz uma chamada de sistema para ler ou escrever um arquivo de disco, ele fornece uma indicação de qual arquivo ele quer, um ponteiro para o buffer de dados e o contador. O controle é então transferido para o sistema operacional, que chama o driver apropriado. Suponha que o driver começa o disco e termina quando ocorre uma interrupção. No caso da leitura do disco, obviamente quem chamou terá de ser bloqueado (pois não há dados para ele). E quanto a escrever para o disco? Quem chamou precisa ser bloqueado esperando o término da transferência de disco?